

Guía de Estudio — Aspectos Avanzados de Programación Funcional

Tema 04 — Estudio autónomo

Matías Gel

Ciclo lectivo 2026

Índice de Contenidos

1	Guía de Estudio — Tema 04	1
1.1	Aspectos Avanzados de Programación Funcional	1
1.2	Cómo usar esta guía	2
1.3	Objetivos de Aprendizaje	2
1.4	Conceptos Previos	2
1.5	Desarrollo Teórico	3
1.5.1	Bloque 1 — Fundamentos avanzados (F-01 a F-13)	3
1.5.2	Bloque 2 — Abstracciones y efectos (F-14 a F-24)	4
1.5.3	Bloque 3 — Concurrencia y metaprogramación (F-25 a F-34)	7
1.5.4	Bloque 4 — Taller comparativo (F-35 a F-42)	9
1.6	Ejemplos Trabajados	10
1.6.1	Ejemplo 1 — Pipeline de órdenes en TypeScript	10
1.6.2	Ejemplo 2 — <code>Result</code> para validación	10
1.6.3	Ejemplo 3 — Versión Clojure orientada a datos	10
1.7	Puntos Clave	11
1.8	Autoevaluación	11
1.8.1	Preguntas de comprensión	11
1.8.2	Ejercicio de aplicación	11
1.9	Glosario	13
1.10	Referencias y Trazabilidad	13

1 Guía de Estudio — Tema 04

1.1 Aspectos Avanzados de Programación Funcional

Materia: Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI — IF020

Año: 4° Licenciatura en Sistemas **Semana:** 3 — Clase 1 **Duración estimada de**

estudio autónomo: 4-5 horas

1.2 Cómo usar esta guía

Esta guía está pensada para estudio autónomo y seguimiento de clase. Se basa en el alcance aprobado en [diseno.md](#), la secuencia didáctica de [minuta.md](#) y el mapa visual de [filminas.md](#).

Secuencia recomendada:

1. Leer objetivos y conceptos previos.
2. Estudiar cada bloque teórico con sus ejemplos en TypeScript y Clojure.
3. Resolver los ejercicios trabajados y luego la autoevaluación.
4. Repasar glosario y errores frecuentes antes de pasar al TP.

1.3 Objetivos de Aprendizaje

Al finalizar, deberías poder:

#	Nivel Bloom	Objetivo
OA-1	Comprender	Comparar patrones funcionales avanzados en TypeScript y Clojure
OA-2	Aplicar	Construir pipelines con <code>map</code> , <code>filter</code> , <code>reduce</code> y composición
OA-3	Aplicar	Modelar errores con <code>Result</code> y ausencias con <code>Option</code> / <code>Maybe</code>
OA-4	Analizar	Distinguir pipeline convencional vs transducers
OA-5	Analizar	Comparar concurrencia con <code>core.async</code> y con <code>Promise</code> / <code>async-await</code>
OA-6	Crear	Diseñar una mini API funcional con genéricos y funciones de orden superior

1.4 Conceptos Previos

Antes de empezar, verificá que puedas explicar con tus palabras:

- Función pura e inmutabilidad.
- Diferencia entre transformación de datos y mutación de estado.

- Operadores básicos en TypeScript (`map`, `filter`, `reduce`).
- Lectura básica de sintaxis Clojure (`def`, `defn`, vectores, mapas).

Si alguno de estos puntos no está sólido, repasá primero el Tema 03.

1.5 Desarrollo Teórico

1.5.1 Bloque 1 — Fundamentos avanzados (F-01 a F-13)

1.5.1.1 1.1 Por qué funcional hoy El enfoque funcional gana relevancia por razones de ingeniería:

- Sistemas con concurrencia y multicore penalizan estado mutable compartido.
- La composición reduce complejidad accidental.
- El comportamiento determinista mejora testing y mantenimiento.

Idea clave: No se trata de rechazar el paradigma imperativo, sino de elegir la herramienta conceptual que mejor controla errores para cada problema.

1.5.1.2 1.2 De operaciones sueltas a pipeline En TypeScript, un pipeline combina transformaciones pequeñas encadenando métodos de array:

```
const numeros = [1, 2, 3, 4, 5, 6];
const resultado = numeros
  .filter(n => n % 2 === 0)    // [2, 4, 6]      † nuevo array
  .map(n => n * n)             // [4, 16, 36]     † nuevo array
  .reduce((acc, n) => acc + n, 0); // 56          † escalar
// resultado = 56
// numeros sigue siendo [1, 2, 3, 4, 5, 6]
```

Lectura conceptual:

- `filter(pred)`: devuelve un nuevo array con los elementos donde `pred(elemento)` es `true`. El original no cambia.
- `map(fn)`: devuelve un nuevo array donde cada elemento fue transformado por `fn`. Siempre tiene el mismo tamaño que el original.
- `reduce(fn, inicial)`: colapsa la colección a un único valor partiendo del `inicial` y acumulando con `fn`.

La ventaja no es solo sintáctica: cada paso tiene intención explícita y puede testearse en aislamiento.

Traza mental de reduce:

```
acc=0,  n=4  → 0  + 4  = 4
acc=4,  n=16 → 4  + 16 = 20
acc=20, n=36 → 20 + 36 = 56
```

1.5.1.3 1.3 Inmutabilidad en práctica En vez de cambiar valores existentes, se crean nuevas versiones:

```
const persona = { name: "Ana", age: 28 } as const;
// persona.age = 29; // Error de compilación con `as const`

const cumpleaños = { ...persona, age: 29 }; // nuevo objeto
console.log(persona.age); // 28 + intacto
console.log(cumpleaños.age); // 29
```

Esto evita efectos colaterales difíciles de rastrear, especialmente bajo concurrencia.

Inmutabilidad en arrays:

```
const lista = [1, 2, 3];
// lista.push(4); // muta el array original
const nueva = [...lista, 4]; // crea uno nuevo
const sinUno = lista.slice(1); // [2, 3] - lista intacta
```

Por qué importa en concurrencia:

Si dos threads leen el mismo objeto inmutable simultáneamente, ninguno puede corromper el estado del otro. No se necesitan locks para lecturas.

1.5.1.4 1.4 Clojure: pereza y persistencia Clojure usa estructuras persistentes e inmutables por defecto.

- Las secuencias perezosas evalúan cuando hace falta.
- Las estructuras comparten partes internas entre versiones.
- Se gana seguridad sin pagar siempre copia completa.

1.5.2 Bloque 2 — Abstracciones y efectos (F-14 a F-24)

1.5.2.1 2.1 Algebraic Data Types en TypeScript Los ADT (Algebraic Data Types) combinan tipos para modelar todos los estados posibles de forma explícita. Hay dos variedades fundamentales:

Tipo producto (AND): todos los campos presentes simultáneamente.

```
type Punto = { x: number; y: number }; // x AND y
```

Tipo suma (OR): exactamente una de varias variantes.

```
type Resultado<T, E> =  
  | { status: "ok";    value: T }    // éxito  
  | { status: "error"; error: E }; // fallo
```

Beneficio:

El tipo suma obliga a considerar ambos caminos (éxito y error). El compilador detecta si hay una rama del switch/if sin manejar.

Ejemplo con Shape:

```
type Shape =  
  | { kind: "circle";    radius: number }  
  | { kind: "rect";      width: number; height: number };  
  
const area = (s: Shape): number => {  
  switch (s.kind) {  
    case "circle": return Math.PI * s.radius ** 2;  
    case "rect":   return s.width * s.height;  
    // Si agregamos una nueva variante y no la manejamos,  
    // TypeScript reporta error en tiempo de compilación.  
  }  
};
```

1.5.2.2 2.2 Result vs excepción Con excepciones, el error puede escapar del flujo declarado. Con Result, el error se vuelve dato y el flujo queda visible.

```
const dividir = (a: number, b: number): Resultado<number, string> =>  
  b === 0  
    ? { status: "error", error: "División por cero" }  
    : { status: "ok", value: a / b };
```

1.5.2.3 2.3 Option / Maybe Representa presencia o ausencia de valor sin depender de null o undefined como convención implícita.

```
type Maybe<T> =  
  | { some: true;  value: T }  
  | { some: false };
```

```

const just    = <T>(v: T): Maybe<T> => ({ some: true,  value: v });
const nothing = <T>(): Maybe<T>    => ({ some: false });

// Búsqueda segura:
const buscarUsuario = (id: number): Maybe<{ nombre: string }> =>
  id === 1 ? just({ nombre: "Ana" }) : nothing();

const u = buscarUsuario(99);
if (u.some) {
  console.log(u.value.nombre); // El compilador sabe que .value existe
}
// Sin el `if`, TypeScript no permite acceder a .value

```

Diferencia con Result: | | Maybe<T> | Result<T, E> | |—|—|—| | Caso de fallo | Solo “no hay valor” | Hay razón tipada del error | | Uso típico | Búsquedas, campos opcionales | Operaciones que producen mensajes de error |

Esto reduce errores de acceso y mejora la robustez del pipeline.

1.5.2.4 2.4 Transducers en Clojure Transducer: composición de transformaciones independiente de la colección destino.

Problema que resuelven:

```

;; Pipeline clásico: crea TRES colecciones intermedias
(->> datos
  (filter activo?)      ; coleccion 1
  (filter alto-valor?)  ; coleccion 2
  (map :total))         ; coleccion 3

```

Con transducer: un solo recorrido

```

(def xf
  (comp
    (filter even?)
    (map #(* % %))))

(transduce xf + [1 2 3 4 5 6])
;; Proceso por elemento: 1 (descartado), 2 (pasa)→2²=4,
;;                               3 (descartado), 4 (pasa)→4²=16, ...
;; + acumula: 4 + 16 + 36 = 56

```

```
;; Sin colección intermedia.
```

Ventaja principal:

Evita colecciones intermedias y aumenta reutilización para flujos grandes. El mismo `xf` funciona con `into`, `sequence` o un canal de `core.async`.

Criterio de uso: aplicar cuando el pipeline tiene 3+ pasos y el volumen de datos es real (miles de elementos o más). En iteraciones pequeñas, el pipeline clásico con `->>` es más legible.

1.5.2.5 2.5 Funciones de orden superior y APIs genéricas Una función de orden superior recibe o devuelve funciones. En diseño de APIs funcionales, esto habilita reutilización de lógica.

```
type Transform<T> = (items: T[]) => T[];

const compose = <T>(...fns: Transform<T>[]) =>
  (input: T[]) => fns.reduce((acc, fn) => fn(acc), input);
```

1.5.3 Bloque 3 — Concurrency y metaprogramación (F-25 a F-34)

1.5.3.1 3.1 Concurrency funcional La inmutabilidad reduce condiciones de carrera porque minimiza estado compartido mutable.

En Clojure — cuatro modelos:

```
;; atom: estado simple, actualizaciones atómicas
(def contador (atom 0))
(swap! contador inc)    ; seguro bajo concurrencia
@contador                ; => 1

;; ref + dosync: múltiples valores coordinados (STM)
(def saldo-a (ref 1000))
(def saldo-b (ref 500))
(dosync
  (alter saldo-a - 200)
  (alter saldo-b + 200))
;; saldo-a=800, saldo-b=700 (ambos o ninguno)

;; agent: actualización asíncrona desacoplada
(def log-eventos (agent []))
(send log-eventos conj {:tipo :click :t (System/currentTimeMillis)})
```

```
;; El envío no bloquea; el agent procesa en otro thread

;; core.async: comunicación por canales (CSP)
(require '[clojure.core.async :refer [chan go >! <!]])
(def eventos (chan 10))
(go (>! eventos {:tipo :enter}))
(go (println "Recibió:" (<! eventos)))
```

En TypeScript:

```
// Promise: valor único futuro
const res = await fetch('/api/ordenes'); // pausa hasta recibir
const json = await res.json();

// async/await: escribe código asíncrono secuencialmente
async function procesar(): Promise<number> {
  const datos = await obtenerDatos(); // efecto: I/O
  return datos // lógica pura desde acá
    .filter(d => d.activo)
    .map(d => d.valor)
    .reduce((a, b) => a + b, 0);
}
```

1.5.3.2 3.2 Separar lógica pura de I/O Regla de diseño:

- Núcleo funcional puro: transforma datos, no habla con el mundo externo.
- Capa de efectos: red, archivo, consola, DB.

Esta separación mejora testabilidad y reduce acoplamiento.

Ejemplo práctico:

```
// Mezclado: la lógica pura y el I/O están entrettejidos
async function reporteTotal(id: string): Promise<string> {
  const res = await fetch(`/api/ordenes/${id}`);
  const data = await res.json();
  let total = 0;
  for (const o of data) if (o.activo) total += o.monto;
  return `Total: ${total}`;
}
```



```
// Separado: I/O al inicio, lógica pura como subfunción
const calcularTotal = (ordenes: Orden[]): number => // pura - testeable sin red
  ordenes
    .filter(o => o.activo)
    .map(o => o.monto)
    .reduce((a, b) => a + b, 0);

async function reporteTotal(id: string): Promise<string> { // efectos
  const res = await fetch(`/api/ordenes/${id}`);
  const data = await res.json();
  return `Total: ${calcularTotal(data)}`; // llama a la pura
}

// Para testear calcularTotal, no necesitamos mock de red.
```

1.5.3.3 3.3 Canal vs promesa Comparación rápida:

Modelo	Uso ideal	Naturaleza
core.async	Flujos de eventos continuos	Comunicación por canal
Promise	Operación puntual con resultado único	Resolución futura

1.5.4 Bloque 4 — Taller comparativo (F-35 a F-42)

Objetivo del taller: Resolver el mismo problema en TypeScript y Clojure para identificar ideas comunes y diferencias expresivas.

Problema base:

- Filtrar órdenes válidas.
- Calcular total de montos.
- Reportar errores de validación explícitamente.

Criterio de corrección conceptual:

- Evitar mutación innecesaria.
- Hacer explícito el manejo de error.
- Mantener composición clara y trazable.

1.6 Ejemplos Trabajados

1.6.1 Ejemplo 1 — Pipeline de órdenes en TypeScript

```
type Order = { amount: number; status: "valid" | "invalid" };

const totalValidas = (orders: Order[]): number =>
  orders
    .filter(o => o.status === "valid")
    .map(o => o.amount)
    .reduce((acc, x) => acc + x, 0);
```

Prueba mental:

- Entrada: [100 valid, 50 invalid, 30 valid]
- Filtrado: [100, 30]
- Reducción: 130

1.6.2 Ejemplo 2 — Result para validación

```
type Resultado<T, E> =
  | { status: "ok"; value: T }
  | { status: "error"; error: E };

type Order = { amount: number; status: "valid" | "invalid" };

const validarOrden = (o: Order): Resultado<number, string> =>
  o.status === "valid"
    ? { status: "ok", value: o.amount }
    : { status: "error", error: "Orden inválida" };
```

Observación: El consumidor de validarOrden debe tratar explícitamente ambos casos.

1.6.3 Ejemplo 3 — Versión Clojure orientada a datos

```
(def orders [{:amount 100 :status :valid}
              {:amount 50 :status :invalid}
              {:amount 30 :status :valid}])

(->> orders
  (filter #(= :valid (:status %))))
```

```
(map :amount)
(reduce +))
; => 130
```

Idea común con TypeScript: Mismo patrón conceptual, distinta sintaxis.

1.7 Puntos Clave

- Funcional avanzado no significa complejidad innecesaria; significa contratos explícitos y composición disciplinada.
 - `Result` y `Option` mejoran legibilidad del flujo de control.
 - Transducers aportan valor cuando hay cadenas de transformación largas o flujos grandes.
 - Concurrencia segura depende más del modelo de datos que de la sintaxis del lenguaje.
-

1.8 Autoevaluación

1.8.1 Preguntas de comprensión

1. ¿Qué problema de mantenimiento reduce la inmutabilidad en sistemas concurrentes?
2. ¿Por qué `Result` mejora trazabilidad frente a excepciones implícitas?
3. ¿Qué diferencia esencial hay entre pipeline convencional y transducer?
4. ¿Cuándo usarías un canal en lugar de una promesa?
5. ¿Cómo separarías lógica pura de I/O en una API pequeña?

1.8.2 Ejercicio de aplicación

Implementá en TypeScript una función que:

- Reciba un array de órdenes.
- Devuelva `Result<number, string>`.
- Si hay alguna orden inválida, devuelva error.
- Si todas son válidas, devuelva total acumulado.

Sugerencia de verificación:

- Caso todo válido.
- Caso con al menos una inválida.
- Caso lista vacía.

Solución guiada (intentá solo primero):

```

type Result<T, E = string> =
  | { ok: true; value: T }
  | { ok: false; error: E };

const ok = <T>(v: T): Result<T, never> => ({ ok: true, value: v });
const err = <E>(e: E): Result<never, E> => ({ ok: false, error: e });

type Orden = { id: number; monto: number; valida: boolean };

const totalOrdenes = (ordenes: Orden[]): Result<number, string> => {
  // Caso lista vacía: es válido, total = 0
  const invalida = ordenes.find(o => !o.valida);
  if (invalida) {
    return err(`Orden ${invalida.id} es inválida`);
  }
  const total = ordenes.reduce((acc, o) => acc + o.monto, 0);
  return ok(total);
};

// Tests mentales:
console.log(totalOrdenes([]));
// => { ok: true, value: 0 }

console.log(totalOrdenes([
  { id: 1, monto: 100, valida: true },
  { id: 2, monto: 200, valida: true },
]));
// => { ok: true, value: 300 }

console.log(totalOrdenes([
  { id: 1, monto: 100, valida: true },
  { id: 2, monto: 50, valida: false },
]));
// => { ok: false, error: "Orden 2 es inválida" }

```

Variante con reduce funcional puro (avanzada):

```

// En vez de `find` + early return, podemos usar reduce:
const totalPuro = (ordenes: Orden[]): Result<number, string> =>

```

```
ordenes.reduce<Result<number, string>>(  
  (acc, o) => {  
    if (!acc.ok) return acc; // propagar error  
    if (!o.valida) return err(`Orden ${o.id} es inválida`);  
    return ok(acc.value + o.monto);  
  },  
  ok(0) // valor inicial: éxito con 0  
);
```

1.9 Glosario

- **Función pura:** función sin efectos colaterales y determinista.
- **Inmutabilidad:** estrategia de no modificar valores existentes.
- **Composición:** combinación de funciones pequeñas para construir una mayor.
- **ADT:** tipo que modela alternativas de estado en forma explícita.
- **Result:** tipo para representar éxito o error como datos.
- **Option / Maybe:** tipo para representar presencia o ausencia de valor.
- **Transducer:** composición de transformaciones independiente de la colección.
- **Laziness:** evaluación diferida hasta que el resultado es necesario.
- **STM:** control transaccional de estado compartido.
- **Canal:** mecanismo de comunicación asíncrona entre productores y consumidores.

1.10 Referencias y Trazabilidad

Trazabilidad a artefactos del tema:

- Diseño aprobado: [diseno.md](#)
- Secuencia docente: [minuta.md](#)
- Soporte visual: [filminas.md](#)

Trazabilidad a filminas clave:

- Fundamentos: F-02 a F-13
- Abstracciones y errores: F-14 a F-24
- Concurrencia y efectos: F-25 a F-34
- Integración práctica y cierre: F-35 a F-42

Nota sobre fuentes PDF: No se detectaron PDFs de referencia en `material/` ni en la carpeta del tema al momento de generar esta guía.